

Session 7 · Deck 7.A4

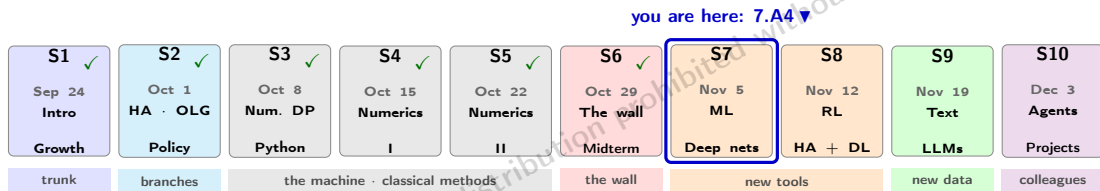
Architecture as a Constraint on W

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 7.A1 ML for macroeconomists · 7.A2 nets as approximators · 7.A3 training · **7.A4** architecture.

One Idea, Three Times

A feedforward layer is $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ with $\mathbf{W}$ **dense**: every entry is a free parameter. That is what makes it general — it assumes nothing about the input — and it is also what makes it expensive, because anything that could have been written into the structure has to be learned from data instead.

Every specialised architecture in this deck is the same layer with a **constraint on $\mathbf{W}$** :

- **CNN** — $\mathbf{W}$ sparse *and* repeated;
- **RNN** — one $\mathbf{W}$ shared across dates;
- **GNN** — the sparsity pattern follows the adjacency.

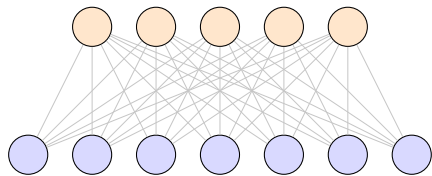
Feng & Miao, *AI for Economists*, ch. 9, "Specialized architectures".

The trade, stated once

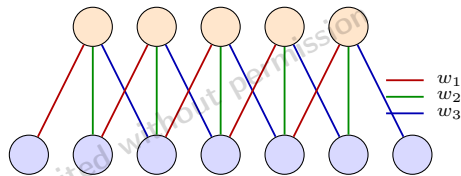
A constraint shrinks the hypothesis space. The **cost** is that the assumption must be true. The **return** is fewer parameters, and so fewer observations needed to pin them down.

This is 7.A2's lesson in another key: a network is a basis you do not choose. An *architecture* is the part of the basis you *do* choose.

Feedforward vs. Convolution



(a) fully connected: $7 \times 5 = 35$ distinct weights



(b) convolution, kernel width 3: only w_1, w_2, w_3

Seven inputs, five outputs. On the left, 35 lines and 35 free parameters. On the right each output sees only **three adjacent** inputs (local connection), and lines of the same colour **share one parameter** (weight sharing): **three** free parameters in the whole layer.

Convolution is not a new operation. It is a feedforward layer after two constraints — sparse, and repeated — are imposed on $\mathbf{W}$.

What the Constraint Buys, and What It Assumes

A 256×256 colour image is 196,608 numbers. A dense first layer into 256 units needs about 5×10^7 weights. Sixty-four 3×3 kernels need

$$64 \times (3 \times 3 \times 3 + 1) = 1,792,$$

about four orders of magnitude fewer. **Fewer parameters means fewer observations** — decisive in economics, where the sample is what it is.

The assumption being bought is that **nearby inputs are related and distant ones are not**. A dense W treats both alike; so would a convolution applied to data with no spatial order at all.

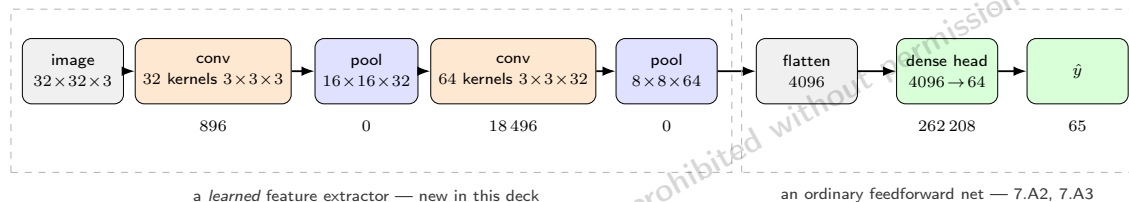
Three things to keep apart

1. Kernels *extract features*; they do not down-sample. With padding and stride 1 the feature map is the same size as the input.
2. A conv layer usually *raises* the channel count: $32 \times 32 \times 3 \rightarrow 32 \times 32 \times 32$.
3. **Pooling** or stride > 1 is what reduces resolution. That is a separate layer.

Weight sharing gives **equivariance**, not invariance: shift the input one pixel and the feature map shifts one pixel. Pooling is what turns that into approximate invariance.

The common slip is $64 \times (9 + 1) = 640$: a colour kernel is $3 \times 3 \times 3$, so 27 weights and a bias. Ledger param_count. Code: [s07_generate.py](#).

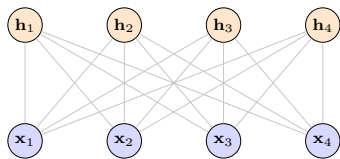
A CNN Is a Feature Extractor Bolted to a Feedforward Net



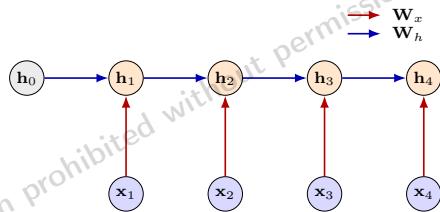
Parameters per layer beneath each block; 281,665 in total. Look at how they are distributed: the two convolutional layers hold 19,392 — **6.9%** — while the dense head after flattening holds 262,273, or **93.1%**.

Convolution saves parameters; the layer after flattening spends them again. And the decomposition is the one to remember: a CNN is h_θ turning an unstructured object into a feature vector, and then the network you already know.

Feedforward vs. Recurrent



(a) feedforward: $4 \times 4 = 16$ distinct weights
change the sequence length and $\mathbf{W}$ changes shape



(b) recurrent: only $\mathbf{W}_x$ and $\mathbf{W}_h$
the count does not depend on T

$$\bullet \mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$$

Unrolled, an RNN over T dates is a T -layer feedforward network whose layers share one $(\mathbf{W}_h, \mathbf{W}_x)$ — so 7.A3's backpropagation applies unchanged. Retreating one date multiplies by $\mathbf{W}_h$; retreating T multiplies by $\mathbf{W}_h^T$.

The Hidden State Is a State Variable

$\mathbf{h}_t = \sigma(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b})$ is a recursion you have written before. It is a **state-space model**: $\mathbf{h}_t$ carries everything in the history up to t that matters for the future, and **training the network is estimating the transition function**.

$\mathbf{W}_h^T$ is then the whole difficulty: spectral radius below one and the gradient vanishes, above one and it explodes. Either way a dependence spanning dozens of dates cannot be learned — monthly data forgets last year.

LSTM keeps a cell state beside $\mathbf{h}_t$ and gates it:

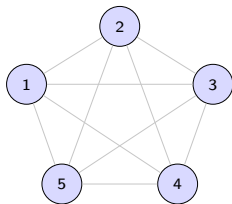
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

The forget gate, read as an economist

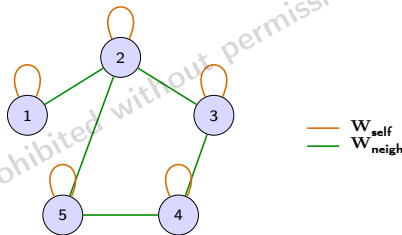
When $\mathbf{f}_t \approx 1$ the old information passes through intact, and the gradient travels along $\mathbf{c}$ almost undamped. In an AR(1), ρ is a **constant**. The forget gate makes persistence a **function of the current state** — long memory for a regime change, quick forgetting for ordinary noise.

Transformers have since overtaken recurrent models (S9). The recursion is still worth knowing — it is the background against which attention is an improvement.

Feedforward vs. Graph



(a) fully connected: n^2 distinct weights
renumber the nodes and the weights must be permuted



(b) graph convolution: only W_{self} and W_{neigh}
all five nodes share the same two

Read the five nodes as five banks and the edges as lending. Flattening the adjacency matrix into a feedforward layer costs n^2 parameters — and worse, the parameterisation depends on the *numbering*, which carries no economic content. Graph convolution passes information only along edges that exist, with one $(W_{\text{self}}, W_{\text{neigh}})$ for every node. Kipf & Welling

(2017). Feng & Miao, *AI for Economists*, ch. 9, Fig. 9.12.

Permutation Invariance Is an Economic Restriction

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}_{\text{self}}\mathbf{h}_v^{(\ell)} + \sum_{u \in \mathcal{N}(v)} \frac{1}{|\mathcal{N}(v)|} \mathbf{W}_{\text{neigh}}\mathbf{h}_u^{(\ell)} + \mathbf{b}\right)$$

Neighbours come in no particular order, so the aggregation **must** be order-free — a sum or a mean. That is not a technical detail. It is the constraint that the answer cannot depend on how the nodes were numbered.

And it shows the family resemblance: on a regular grid each pixel's neighbours are a fixed k , so the shared weights can be split by position into w_1, w_2, w_3 . **Convolution is the special case of this construction on a grid.**

Where you have met this already

A heterogeneous-agent model requires prices to depend on the distribution **only through statistics that do not depend on which household is which**. That is the same restriction as permutation invariance — and it is why a distribution can be fed to a network at all. **S8 builds on exactly this.**

L layers let each node see its L -hop neighbourhood. Depth on a graph is *reach*.

Choosing the Constraint

Data	Architecture	Constraint on W	Typical use in economics
tabular	feedforward	none; W dense	consumption forecasting, policy-function approximation , treatment effects
images, spatial	CNN	sparse (local) <i>and</i> repeated (shared)	satellite imagery, street view, spatial patterns
sequences	RNN, LSTM	one W across dates; gates make persistence state-dependent	macro time-series forecasting, text sentiment
networks, graphs	GNN	sparsity follows adjacency; aggregation order-free	financial-network risk, supply-chain transmission, spillovers

The boundaries are not sharp and mixtures are normal — a CNN makes a feature vector, which then joins tabular variables in a feedforward net. **When the assumption holds:** fewer parameters, less data, better extrapolation. **When it fails:** the constraint is simply bias.

What These Architectures Do Not Promise

The inductive bias may be doing the work, not the depth. Rolf et al. (2021) generate convolutional kernels **at random**, never train them, and ridge-regress on the resulting features — matching end-to-end deep training across a range of tasks. What pays is locality and sharing.

The sequence evidence is mixed. Paranhos (2025) finds recurrent networks “competitive but not dominant” for US inflation; Kılıç (2025) finds threshold and smooth-transition HAR beating LSTM and GRU on realised volatility. Short, noisy macro-financial series are not automatically deep-model territory. The positive case is Barkan et al. (2023): a hierarchical RNN following the CPI’s own component tree — **structure, not scale**.

Rolf et al. (2021), *Nature Communications* (MOSAICS); Paranhos (2025) and Barkan et al. (2023), *IJF*; Kılıç (2025).

Graphs: an honest reading

There is as yet **no mature GNN application in a mainstream economics journal**; the work sits in CS and quantitative-finance preprints. Production-network transmission is still done with structural models and input–output tables, because identification comes from an explicit shock — and **the network is endogenous**: whatever made two firms trade may also set how they respond.

Takeaways, and Where the Fourth Constraint Lives

- Every architecture here is $\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$ with a **restriction on $\mathbf{W}$** : sparse and repeated (CNN), shared across dates (RNN), following the adjacency (GNN).
- A restriction is a **prior about the target function**, not a trick. True, it buys parameters and samples; false, it buys bias.
- A CNN is a learned feature extractor plus the feedforward net of 7.A2–7.A3 — and the feedforward part still holds **93.1%** of the parameters.
- Order-free aggregation is the same requirement as **prices depending on a distribution only through index-free statistics**.

Bridge to Block B

There is a **fourth** class of constraint, and it is the one economics cares about most: restrict the **signs** of the weights so the network is monotone, or convex, by construction. That is not a fact about images or sequences. It is a theorem about preferences and value functions, written into the parameterisation — and it is **7.B3**.

7.B1 comes first: write a macro model as a loss function, and let the network of 7.A2 approximate its policy.